

Combinational & Sequential Logic Circuits

Presented by: Miss.R.V.Dhage

Department of Physics and Electronics

Rajarshi Shahu

Mahavidyalaya(Autonomous),Latur

INTRODUCTION

- In digital **circuit** theory, **sequential logic** is a type of **logic circuit** whose output depends not only on the present value of its input signals but on the sequence of past inputs, the input history as well. This is in contrast to **combinational logic**, whose output is a function of only the present input.

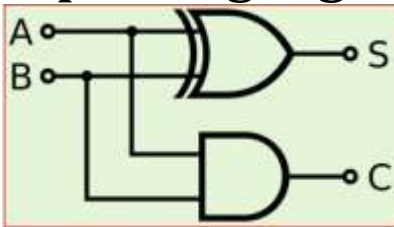
- **Difference between combinational and sequential circuit. ...**
- **Sequential circuits** are those which are dependent on clock cycles and depends on present as well as past inputs to generate any output.
- **Combinational Circuit –**
- In this output depends only upon present input.
- The sequential logic has memory while combinational logic does not.
- They employ a feedback loop to give output back to input.
- **Sequential logic circuits** is a form of binary **circuit**; its design employs one or more inputs and one or more outputs.

- A **combinational logic circuit** performs an operation assigned logically by a **Boolean** expression or truth table. Examples of common **combinational logic circuits** include: half adders, full adders, **multiplexers**, **demultiplexers**, encoders and decoders .

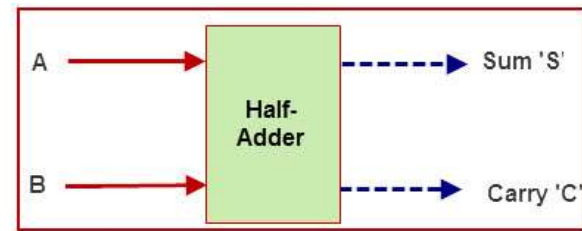
- A **Sequential logic circuits** is a form of binary **circuit**; its design employs one or more inputs and one or more outputs, whose states are related to some definite rules that depends on previous states. ... **Examples** of such **circuits** include clocks, flip-flops, bi-stables, counters, memories, and registers.
- There are two **types of sequential circuit**, synchronous and asynchronous. Synchronous **types** use pulsed or level inputs and a clock input to drive the **circuit**(with restrictions on pulse width and **circuit** propagation). Asynchronous **sequential circuits** do not use a clock signal as synchronous **circuits** do.
- **Flip flop** is a **sequential circuit** which generally samples its inputs and changes its outputs only at particular instants of time and not continuously. **Flip flop** is said to be edge sensitive or edge triggered rather than being level triggered like latches.

Half adder

- An **adder** is a digital circuit that performs addition of numbers. Half adder has only two inputs and two outputs. The **half adder** adds **two binary digits** called as augend and addend and produces two outputs as **sum and carry**; XOR is applied to both inputs to produce sum and AND gate is applied to both inputs to produce carry.
- By using half adder, you can design simple addition with the help of logic gates.



Circuit Implementation



Block Diagram

Truth Table

INPUTS		OUTPUTS	
A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Why it is called as Half-adder ?

- The **half adder** can add only two input bits (A and B) and has nothing to do with the carry if there is any in the input. So if the input to a **half adder** have a carry, then it will neglect it and adds only the A and B bits. **That means the binary addition process is not complete and that's why it is called a half adder**

How Half Adder works ?

- **Half adder** is a simple combinational circuit used to add two single bits. It accepts two inputs and produce two outputs that is a sum output and a carry output. A **half adder** consists of two logic gates 1) XOR and 2) AND gate. And the carry operation performed by AND gate thus carry out put **will** be $A+B$.

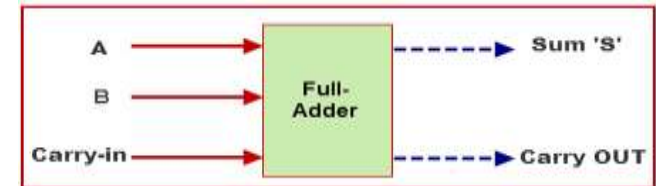
FULL ADDER

The full adder adds 3 one bit numbers, where two can be referred to as **operands** and **one can be referred to as bit carried in**. It produces 2-bit output and these can be referred to as output carry and sum.

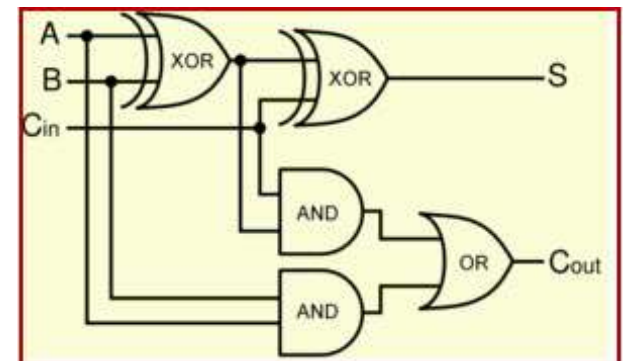
The full-adder has **three inputs and two outputs**. The first two inputs are A and B and the third input is an input carry as C-IN. When a full-adder logic is designed, you string eight of them together to create a byte-wide adder and cascade the carry bit from one adder to the next.

Truth Table

INPUTS			OUTPUT	
A	B	C-IN	C-OUT	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

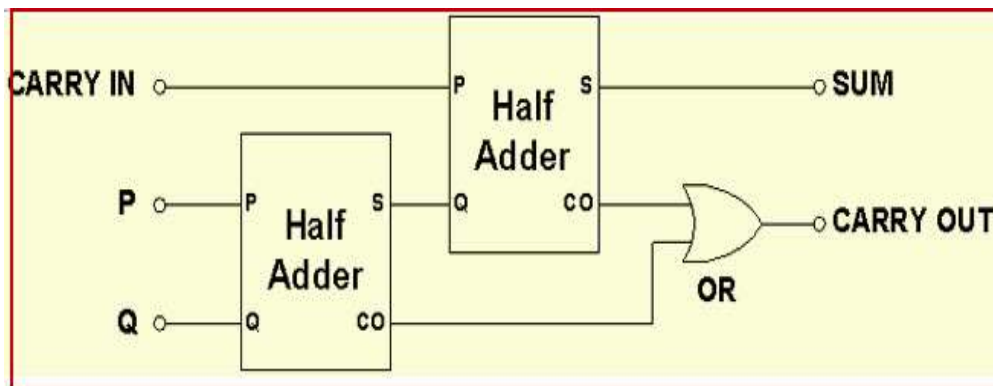


Block Diagram

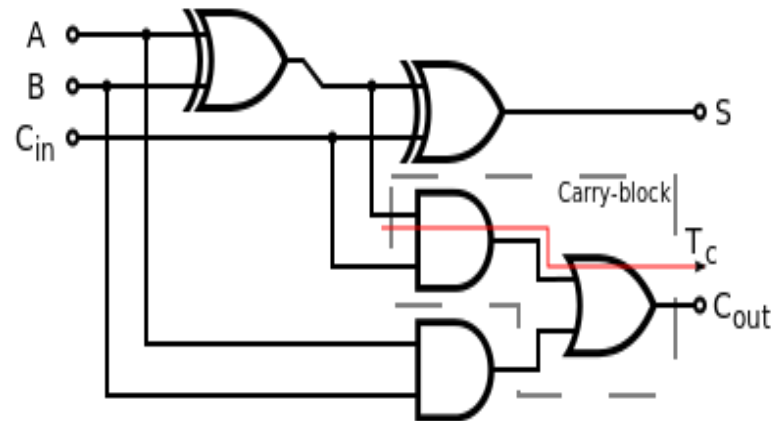


Circuit Implementation

- With the truth-table, the full adder logic can be implemented. You can see that the output S is an XOR between the input A and the half-adder, SUM output with B and C-IN inputs. We take **C-OUT will only be true if any of the two inputs out of the three are HIGH.**
- So, we can implement a full adder circuit with the help of two half adder circuits. **At first, half adder will be used to add A and B to produce a partial Sum and a second half adder logic can be used to add C-IN to the Sum produced by the first half adder to get the final S output.**
- If any of the half adder logic produces a carry, there will be an output carry. So, C-OUT will be an OR function of the half-adder Carry outputs.
- The implementation of larger logic diagrams is possible with the above full adder logic a simpler symbol is mostly used to represent the operation. Given below is a simpler schematic representation of a full adder.

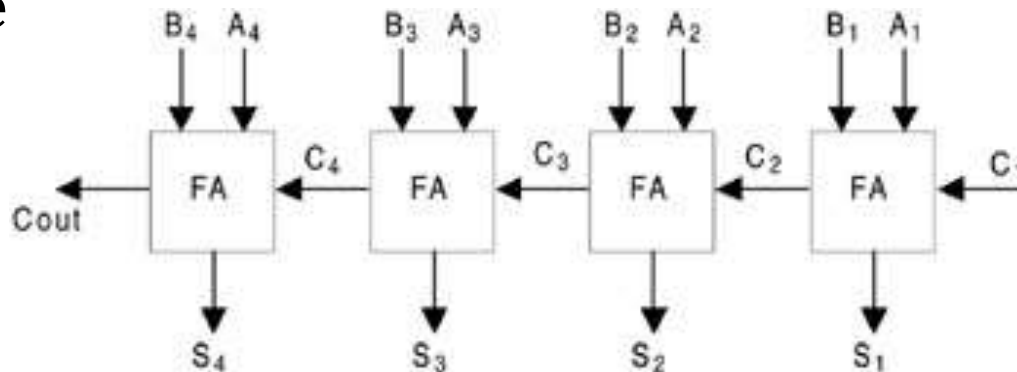


- A **full adder** is a digital **circuit** that performs addition. **Full adders** are implemented with logic gates in hardware. A **full adder** adds three one-bit binary numbers, two operands and a carry bit. The **adder** outputs two numbers, a sum and a carry bit.
- The **Boolean expression for a full adder** is as follows.
- For the CARRY-OUT (C_{out}) bit: $CARRY-OUT = A \text{ AND } B \text{ OR } C_{in}(A \text{ XOR } B) = A.B + C_{in}(A \oplus B)$



4-Bit Parallel Binary adder :

- A **binary parallel adder** is a digital function that produces the arithmetic sum of two **binary** numbers in **parallel**.
- **FOUR-BIT BINARY PARALLEL ADDER** is a circuit in which two binary numbers each of n **bits** can be added by means of a full **adder** circuit. Consider the example that two **4-bit** binary numbers $B_4B_3B_2B_1$ and $A_4A_3A_2A_1$ are to be added with a carry input C_1 .
- A group of four bits is called a **nibble**. A basic 4-bit parallel adder is implemented with four full-adder stages as shown in Figure



- Again, the LSBs (A_1 and B_1) in each number being added go into the right-most full-adder: the higher-order bits are applied as shown to the successively higher-order adders, with the MSBs (A_4 and B_4) in each number being applied to the left-most full-adder. The carry output of each adder is connected to the carry input of the next higher-order adder as indicated. These are called internal carries. In keeping with most manufacturers' data sheets, the input labeled C , is the input carry to the least significant bit adder; C_4 in the case of four bits, is the output carry of the most significant bit adder; and $\Sigma 1$ (LSB) through $\Sigma 4$ (MSB) are the sum outputs. The logic symbol for 4-bit parallel adder is shown in Figure.
- Two binary numbers each of n bits can be added by means of a full adder circuit. Consider the example that two 4-bit binary numbers $B_4B_3B_2B_1$ and $A_4A_3A_2A_1$ are to be added with a carry input C_1 . This can be done by cascading four full adder circuits as shown in figure. The least significant bits A_1 , B_1 , and C_1 are added to produce sum output S_1 and carry output C_2 . Carry output C_2 is then added to the next significant bits A_2 and B_2 producing sum output S_2 and carry output C_3 . C_3 is then added to A_3 and B_3 and so on. Thus finally producing the four-bit sum output $S_4S_3S_2S_1$ and final carry output C_{out} . Such type of four-bit binary adder is commercially available in an IC package.

Half Subtractor :

- A half subtractor is an arithmetic circuit that subtracts two bits and produces their difference. The circuit has two inputs minuend (X) and subtrahend (Y) and two output bits, one is the difference bit (D) and the other is the borrow bit (B).
- As like addition operation of 2 binary digits, which produces SUM and CARRY, the subtraction of 2 binary digits also produces two outputs which are termed as **difference and borrow**. The simplest possible subtraction of 2-bit binary digits consists of four possible operations, they are **0-0**, **0-1**, **1-0** and **1-1**. The operations **0-0**, **1-0** and **1-1** produces a subtraction of 1-bit output whereas, the remaining operation **0-1** produces a 2-bit output. They are referred as **difference** and **borrow** bit respectively. This **borrow** bit is used for subtraction of the next higher pair bit.
- So, we can define half subtractor as a combinational circuit which is capable of performing subtraction of 2-bit binary digits is known as a half subtractor. Here, the binary digit from which the other digit is subtracted is called **minuend** and the binary digit which is to be subtracted is known as the **subtrahend**.

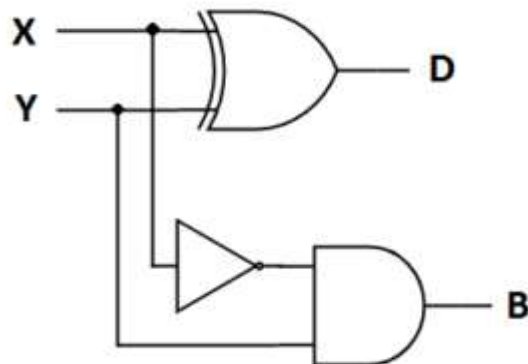
- It performs the operation $X - Y$. It should be noted that the weight of the output borrow bit is -2, while the weight of the output difference bit is +1.
- The truth table of the half subtractor is shown. The Boolean functions for the two outputs can be obtained directly from the truth table as:

$$D = (XY + XY) = X \oplus Y$$

- **The half subtractor boolean expressions are :**

- **$D = (X'Y + XY') = X \oplus Y$**

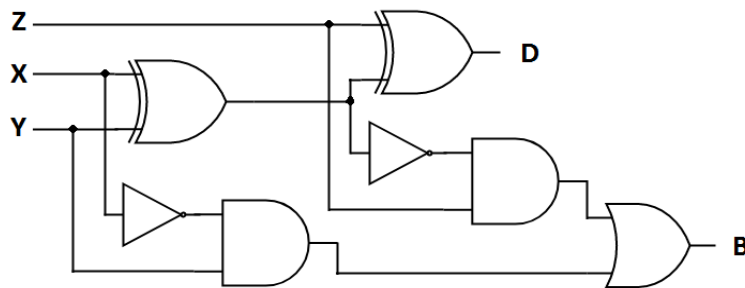
- **$B = X'Y$**



Inputs		Outputs	
X	Y	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Full Subtractor:

- A full subtractor is a combinational circuit that performs a subtraction between two bits, taking into account that a 1 may have been borrowed by a lower significant bit. The circuit has three inputs and two outputs.
- Input variables are minuend (X), subtrahend (Y), and previous borrow (Z); output variables are difference (D) and output borrow (B).
- It performs the operation $X - Y - Z$. It should be noted that the weight of the output borrow bit is -2, while the weight of the output difference bit is +1. The truth table of the full subtractor is shown.
- **The full subtractor boolean expressions are :**
- $(X'Y'Z + X'YZ' + XY'Z' + XYZ) = X \oplus Y \oplus Z$
- $(X'Y'Z + X'YZ' + X'YZ + XYZ) = X'(Y \oplus Z) + YZ$



Inputs			Outputs	
X	Y	Z	D	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

- When there is a situation where the minuend and subtrahend number contains more significant bit, then the **borrow** bit which is obtained from the subtraction of 2-bit binary digits is subtracted from the next higher order pair of bits. In such situation, the subtraction involves the operation of 3 bits. Such situation of subtraction can't handle by a simple half subtractor. So, combining two half subtractor we can form another combinational circuit which can perform this type of operation. This circuit is known as the full subtractor.
- So we can define full subtractor as a combinational circuit which takes three inputs and produces two outputs **difference** and **borrow**. Above is the truth table of the full subtractor, we have used three input variables X, Y and Z which refers to the term **minuend**, **subtrahend** and **borrow** bit respectively. The two outputs **difference** and **borrow** are named as D and B respectively.
- The construction of **full subtractor circuit diagram** involves two half subtractor joined by an OR gate as shown in the above **circuit diagram of the full subtractor**. The two borrow bits generated by two separate half subtractor are fed to the OR gate which produces the final borrow bit. The final difference bit is the combination of the difference output of the first half adder and the next higher order pair of bits.

Sequential Logic Circuits :

FLIP-FLOPS :

- Both **Latches and flip flops** are circuit elements wherein the output not only depends on the current inputs, but also depends on the previous input and outputs. The main **difference between the latch and flip flop** is that a **flip flop** has a clock signal, whereas a **latch** does not.
- A **flip-flop** or **latch** is a circuit that has two stable states and can be used to **store state information**. A **flip-flop** is a bistable multivibrator. The circuit can be made to change state by signals applied to one or more control inputs and will have one or two outputs.
- Latches and flip-flops are the basic elements for storing information. One latch or **flip-flop can store one bit of information**. The main difference between latches and flip-flops is that for latches, their outputs are constantly affected by their inputs as long as the enable signal is asserted. In other words, **when they are enabled, their content changes immediately when their inputs change**. Flip-flops, on the other hand, **have their content change only** either at the rising or falling edge of the enable signal. This enable signal is usually the controlling **clock signal**. After the rising or falling edge of the clock, the flip-flop content remains constant even if the input changes.

- There are basically four main types of latches and flip-flops:
- **SR, D, JK, and T.** The major differences in these flip-flop types are the number of **inputs they have and how they change state**. For each type, there are also different variations that enhance their operations.
- Each type can have different variations such as active high or low inputs, whether they change state at the rising or falling edge of the clock signal, and whether they have asynchronous inputs or not. The flip-flops can be described fully and uniquely by its logic symbol, characteristic table, characteristic equation, state diagram, or excitation table, and are summarized in Figure below.

Flip-Flops : SR, D, JK, and T Flip-Flops

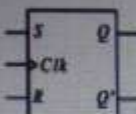
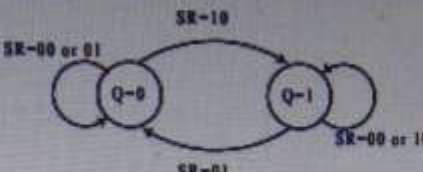
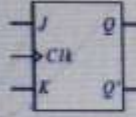
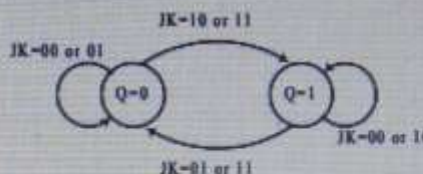
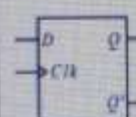
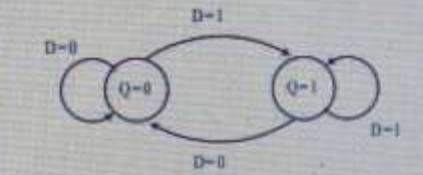
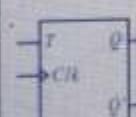
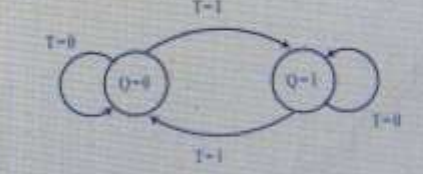
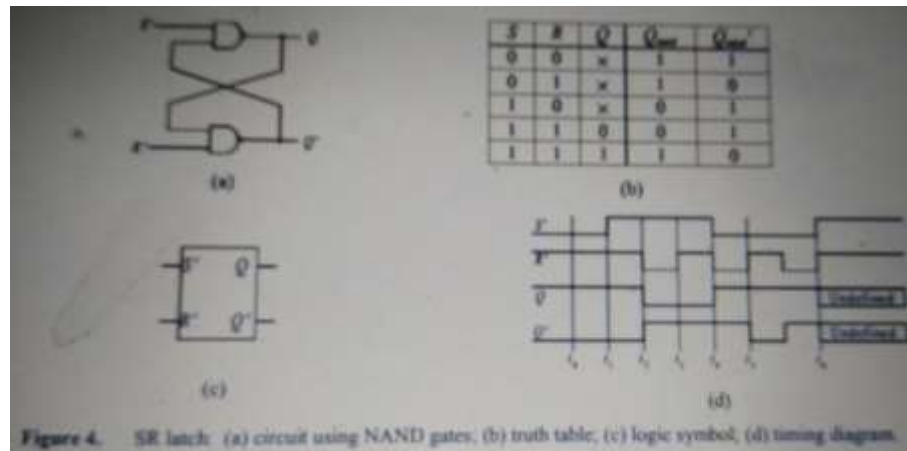
Name / Symbol	Characteristic (Truth) Table	State Diagram / Characteristic Equations	Excitation Table																																																								
SR 	<table border="1"> <thead> <tr> <th>S</th><th>R</th><th>Q</th><th>Q_{next}</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td><td>×</td></tr> <tr><td>1</td><td>1</td><td>1</td><td>×</td></tr> </tbody> </table>	S	R	Q	Q _{next}	0	0	0	0	0	0	1	1	0	1	0	0	0	1	1	0	1	0	0	1	1	0	1	1	1	1	0	×	1	1	1	×	 $Q_{next} = S + R'Q$ $SR = 0$	<table border="1"> <thead> <tr> <th>Q</th><th>Q_{next}</th><th>S</th><th>R</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td><td>×</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>×</td><td>0</td></tr> </tbody> </table>	Q	Q _{next}	S	R	0	0	0	×	0	1	1	0	1	0	0	1	1	1	×	0
S	R	Q	Q _{next}																																																								
0	0	0	0																																																								
0	0	1	1																																																								
0	1	0	0																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	0	1	1																																																								
1	1	0	×																																																								
1	1	1	×																																																								
Q	Q _{next}	S	R																																																								
0	0	0	×																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	1	×	0																																																								
JK 	<table border="1"> <thead> <tr> <th>J</th><th>K</th><th>Q</th><th>Q_{next}</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	J	K	Q	Q _{next}	0	0	0	0	0	0	1	1	0	1	0	0	0	1	1	0	1	0	0	1	1	0	1	1	1	1	0	1	1	1	1	0	 $Q_{next} = J'K'Q + JK' + JKQ'$ $= J'K'Q + JK'Q + JK'Q' + JKQ'$ $= K'Q(J' + J) + JQ'(K' + K)$ $= K'Q + JQ'$	<table border="1"> <thead> <tr> <th>Q</th><th>Q_{next}</th><th>J</th><th>K</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td><td>×</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>×</td></tr> <tr><td>1</td><td>0</td><td>×</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>×</td><td>0</td></tr> </tbody> </table>	Q	Q _{next}	J	K	0	0	0	×	0	1	1	×	1	0	×	1	1	1	×	0
J	K	Q	Q _{next}																																																								
0	0	0	0																																																								
0	0	1	1																																																								
0	1	0	0																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	0	1	1																																																								
1	1	0	1																																																								
1	1	1	0																																																								
Q	Q _{next}	J	K																																																								
0	0	0	×																																																								
0	1	1	×																																																								
1	0	×	1																																																								
1	1	×	0																																																								
D 	<table border="1"> <thead> <tr> <th>D</th><th>Q</th><th>Q_{next}</th></tr> </thead> <tbody> <tr><td>0</td><td>×</td><td>0</td></tr> <tr><td>1</td><td>×</td><td>1</td></tr> </tbody> </table>	D	Q	Q _{next}	0	×	0	1	×	1	 $Q_{next} = D$	<table border="1"> <thead> <tr> <th>Q</th><th>Q_{next}</th><th>D</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </tbody> </table>	Q	Q _{next}	D	0	0	0	0	1	1	1	0	0	1	1	1																																
D	Q	Q _{next}																																																									
0	×	0																																																									
1	×	1																																																									
Q	Q _{next}	D																																																									
0	0	0																																																									
0	1	1																																																									
1	0	0																																																									
1	1	1																																																									
T 	<table border="1"> <thead> <tr> <th>T</th><th>Q</th><th>Q_{next}</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	T	Q	Q _{next}	0	0	0	0	1	1	1	0	1	1	1	0	 $Q_{next} = TQ' + T'Q = T \oplus Q$	<table border="1"> <thead> <tr> <th>Q</th><th>Q_{next}</th><th>T</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	Q	Q _{next}	T	0	0	0	0	1	1	1	0	1	1	1	0																										
T	Q	Q _{next}																																																									
0	0	0																																																									
0	1	1																																																									
1	0	1																																																									
1	1	0																																																									
Q	Q _{next}	T																																																									
0	0	0																																																									
0	1	1																																																									
1	0	1																																																									
1	1	0																																																									

Figure 15. Flip-flop types

Figure 15. Flip-flop types

S-R Latch using NAND & NOR Gate :

- The bistable element is able to remember or store one bit of information. However, because it does not have any inputs, we cannot change the information bit that is stored in it. In order to change the information bit, we need to add inputs to the circuit. The simplest way to add inputs is to replace the two inverters with two NAND gates as shown in Figure. This circuit is called a **SR latch**.
- In addition to the two outputs Q and Q' , there are two inputs S' and R' for set and reset respectively. Following the convention, the prime in S and R denotes that these inputs are active low. The SR latch can be in one of two states: **a set state when $Q = 1$, or a reset state when $Q = 0$.**



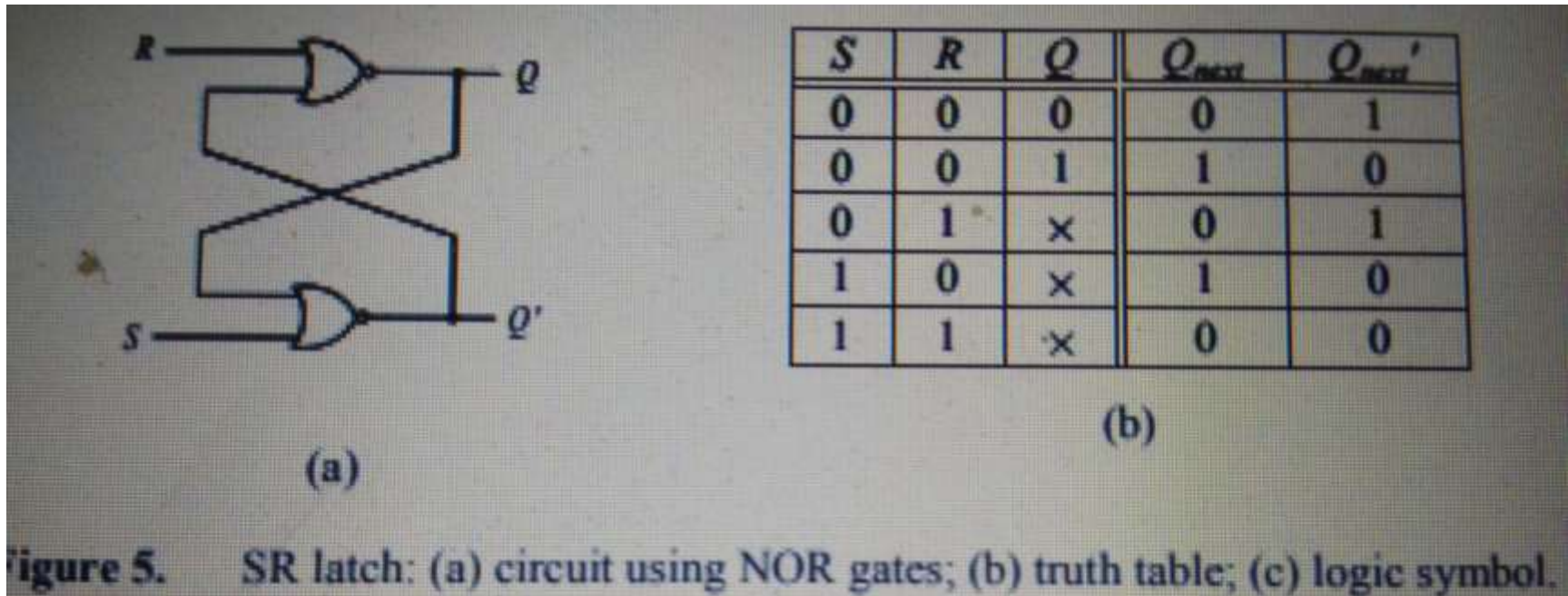
- Figure 4. SR latch: (a) circuit using NAND gates; (b) truth table; (c) logic symbol; (d) timing diagram.

Continued...

- Like the NOR Gate S-R flip flop, this one also has four states. They are
- **S=0, R=1—Q=0, Q'=1**
- This state is also called the **SET** state.
- **S=1, R=0—Q=1, Q'=0**
- This state is known as the **RESET** state.
- In both the states you can see that the outputs are just compliments of each other and that the value of
- Q follows the compliment value of S.
- **S=0, R=0—Q=Q0, & Q' =Q0' No change**
- If both the values of S and R are switched to 0, then the circuit remembers the value of S and R in their previous state.
- **S=1, R=1—Q & Q' = Remember**
- If both the values of S and R are switched to 1 it is an invalid state because the values of both Q and Q' are 1. They are supposed to be compliments of each other. Normally, this state must be avoided.

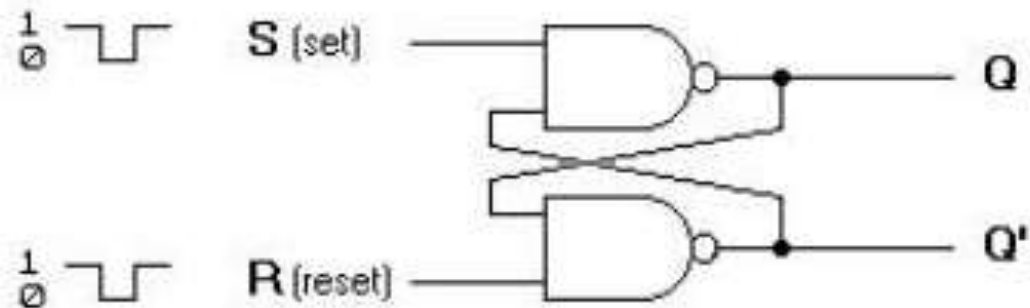
Continued

- Another reason why we do not want both inputs to be asserted i.e. $R=S=1$ is that when they are both asserted, Q is equal to Q' , but we usually want Q to be the inverse of Q' .



- Figure 5. SR latch: (a) circuit using NOR gates; (b) truth table; (c) logic symbol.

S-R FLIP FLOP USING NAND GATE



(a) Logic diagram

S	R	Q	Q'
1	0	0	1
1	1	0	1
0	1	1	0
1	1	1	0
0	0	1	1

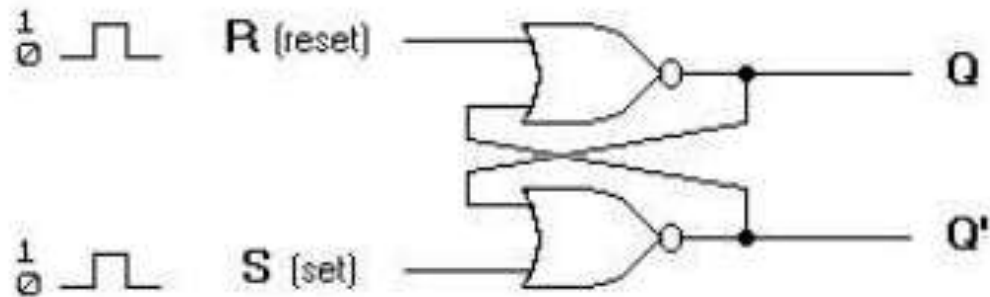
(after S=1, R=0)

(after S=0, R=1)

(b) Truth table

Basic flip-flop circuit with NAND gates

S-R FLIP FLOP USING NOR GATE



(a) Logic diagram

S	R	Q	Q'	
1	0	1	0	
0	0	1	0	(after S=1, R=0)
0	1	0	1	
0	0	0	1	(after S=0, R=1)
1	1	0	0	

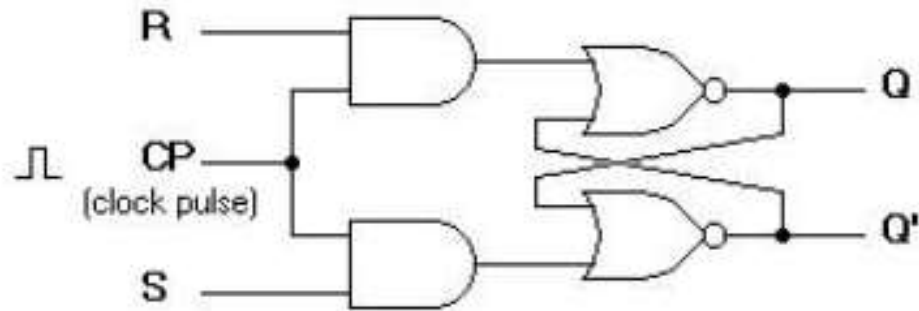
(b) Truth table

Basic flip-flop circuit with NOR gates

Clocked S-R Flip-Flop :

- The problems with S-R flip flops using NOR and NAND gate is the invalid state. This problem can be overcome by using a bistable SR flip-flop that can change outputs when certain invalid states are met, regardless of the condition of either the Set or the Reset inputs. For this, a clocked S-R flip flop is designed by adding two AND gates to a basic NOR Gate flip flop. The circuit diagram and truth table is shown below.
- A clock pulse [CP] is given to the inputs of the AND Gate. When the value of the **clock pulse is '0'**, the **outputs** of both the AND Gates **remain '0'**. As soon as a pulse is given the value of CP turns '1'. This makes the values at S and R to pass through the NOR Gate flip flop. But when the values of both S and R values turn '1', the HIGH value of CP causes both of them to turn to '0' for a short moment. As soon as the pulse is removed, the flip flop state becomes intermediate. Thus either of the two states may be caused, and it depends on whether the set or reset input of the flip-flop remains a '1' longer than the transition to '0' at the end of the pulse. **Thus the invalid states can be eliminated.**

Clocked S-R Flip-flop



(a) Logic diagram

Q	S	R	Q(t+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	indeterminate
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	indeterminate

(b) Truth table

Clocked SR flip-flop

J-K FLIP-FLOP :

- JK flip-flops are very similar to SR flip-flops. The J input is just like the S input in that when asserted, it sets the flip-flop. Similarly, the K input is like the R input where it clears the flip-flop when asserted. **The only difference is when both inputs are asserted. For the SR flip-flop, the next state is undefined, whereas, for the JK flip-flop, the next state is the inverse of the current state. In other words, the JK flip-flop toggles its state when both inputs are asserted.** The circuit, truth table and the logic symbol for the JK flip-flop is shown in Figure

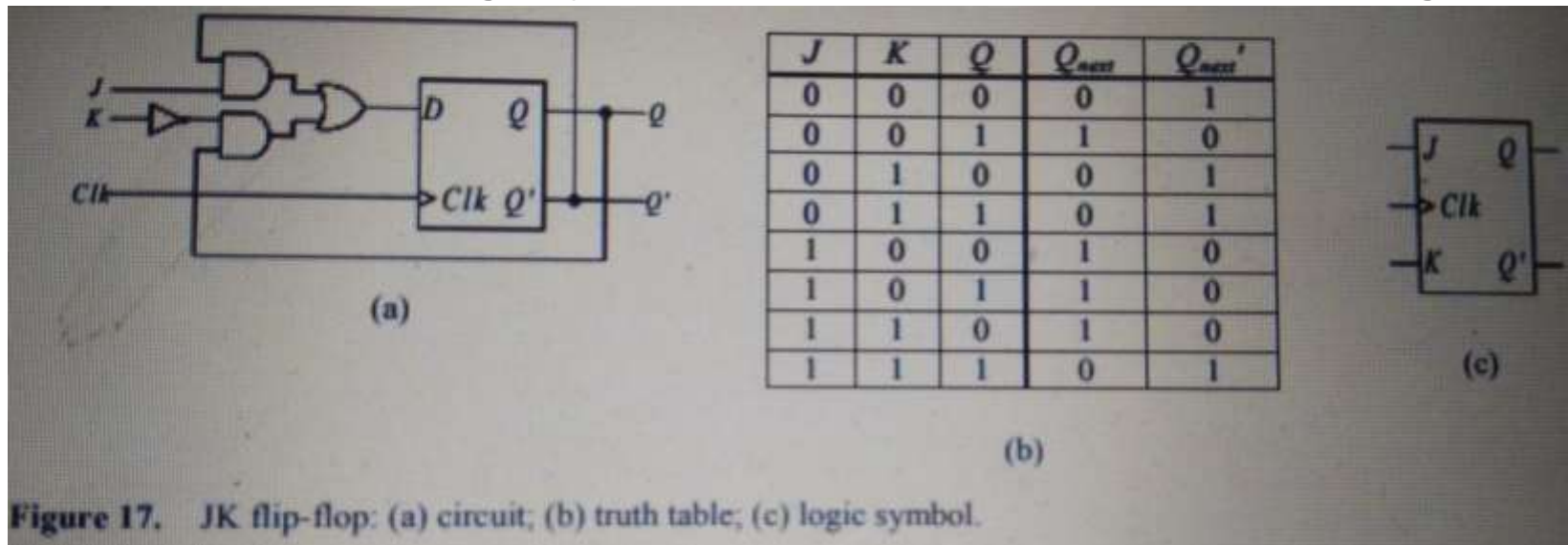
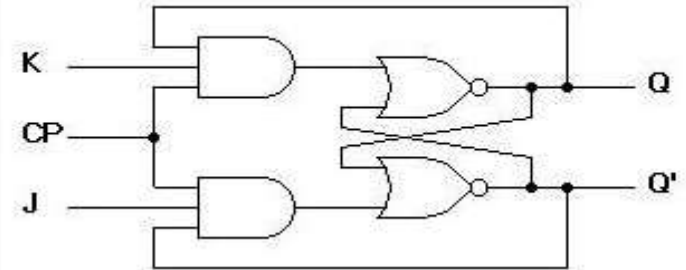


Figure 17. JK flip-flop: (a) circuit; (b) truth table; (c) logic symbol.

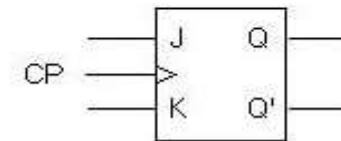
- Figure 17. JK flip-flop: (a) circuit; (b) truth table; (c) logic symbol.

J-K Flip-Flop :

- A J-K flip flop can also be defined as a modification of the S-R flip flop. The only difference is that the intermediate state is more refined and precise than that of a S-R flip flop.
- The behavior of inputs J and K is same as the S and R inputs of the S-R flip flop. The letter J stands for SET and the letter K stands for CLEAR.



(a) Logic diagram



(b) Graphical symbol

Q	J	K	Q(t+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

(c) Transition table

Clocked JK flip-flop

J-K Flip-Flop :

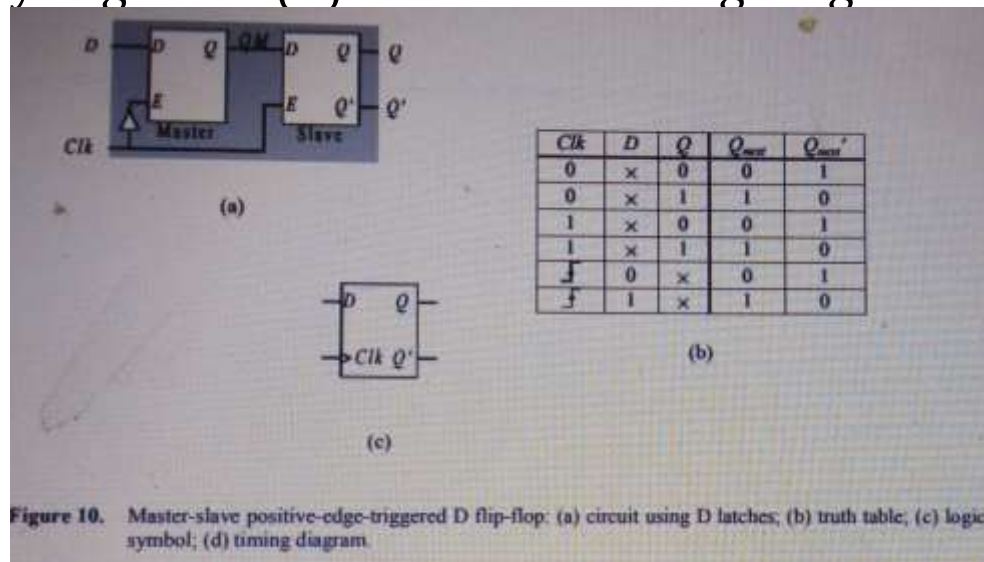
- When both the inputs J and K have a HIGH state, the flip-flop switch to the complement state. So, for a value of $Q = 1$, it switches to $Q=0$ and for a value of $Q = 0$, it switches to $Q=1$.
- The circuit includes two 3-input AND gates. The output Q of the flip flop is returned back as a feedback to the input of the AND along with other inputs like K and clock pulse [CP]. So, if the value of CP is 1, the flip flop gets a CLEAR signal and with the condition that the value of Q was earlier 1. Similarly output Q' of the flip flop is given as a feedback to the input of the AND along with other inputs like J and clock pulse [CP]. So the output becomes SET when the value of CP is 1 only if the value of Q' was earlier 1.
- **The output may be repeated in transitions once they have been complimented for $J=K=1$ because of the feedback connection in the JK flip-flop. This can be avoided by setting a time duration lesser than the propagation delay through the flip-flop. The restriction on the pulse width can be eliminated with a master-slave or edge-triggered construction.**

D TYPE FLIP-FLOP :

- Latches are often called level-sensitive because their output follows their inputs as long as they are enabled. They are transparent during this entire time when the enable signal is asserted. There are situations when it is more useful to **have the output change only at the rising or falling edge of the enable signal**. This enable signal is usually the controlling clock signal. Thus, we can have all changes synchronized to the rising or falling edge of the clock. An edge-triggered flip-flop achieves this by combining in series a pair of latches. Figure shows a positive edge-triggered D flip-flop where two D latches are connected in series and a clock signal Clk is connected to the E input of the latches, one directly, and one through an inverter. The first latch is called the master latch. The master latch is enabled when $\text{Clk} = 0$ and follows the primary input D. When Clk is a 1, the master latch is disabled but the second latch, called the slave latch, is enabled so that the output from the master latch is transferred to the slave latch. The slave latch is enabled all the while that $\text{Clk} = 1$, but its content changes only at the beginning of the cycle, that is, only at the rising edge of the signal because once Clk is 1, the master latch is disabled and so the

Continued...

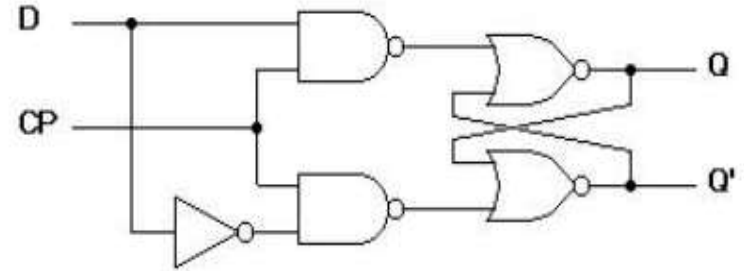
- input to the slave latch will not change. The circuit of Figure 10(a) is called a positive edge-triggered flip-flop because the output Q on the slave latch changes only at the rising edge of the clock. If the slave latch is enabled when the clock is low, then it is referred to as a negative edge-triggered flip-flop. The circuit of Figure 10(a) is also referred to as a masterslave D flip-flop because of the two latches used in the circuit. Figure 10(b) and (c) show the truth table and the logic symbol respectively. Figure 10(d) shows the timing diagram for the D flip-flop.



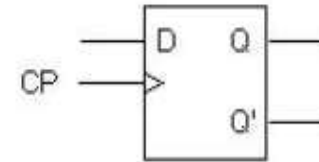
- Figure 10. Master-slave positive-edge-triggered D flip-flop: (a) circuit using D latches; (b) truth table; (c) logic symbol; (d) timing diagram.

D type Flip-Flop :

- The circuit diagram and truth table is given in figure
- D flip flop is actually a slight modification of the above explained **clocked SR flip-flop**. From the figure you can see that the D input is connected to the S input and the complement of the D input is connected to the R input. The D input is passed on to the flip flop when the value of CP is '1'.
- When CP is HIGH, the flip flop moves to the SET state. If it is '0', the flip flop switches to the CLEAR state.



(a) Logic diagram with NAND gates



(b) Graphical symbol

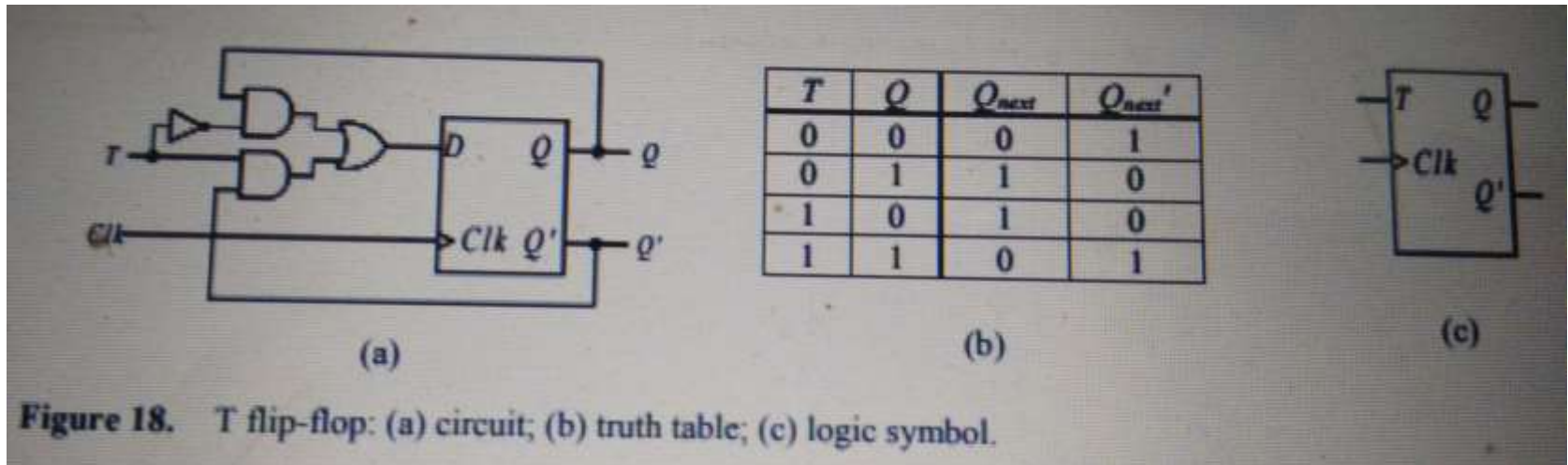
Q	D	Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

(c) Transition table

Clocked D flip-flop

T type Flip-Flop :

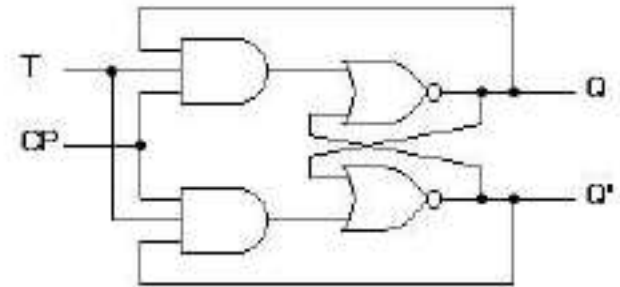
- The T flip-flop has one input in addition to the clock. T stands for **toggle** for the obvious reason. When T is asserted ($T = 1$), the flip-flop state toggles back and forth, and when T is de-asserted, the flip-flop keeps its current state. The T flip-flop can be constructed using a D flip-flop with the two outputs Q and Q' feedback to the D input through a multiplexer that is controlled by the T input as shown in Figure 18.



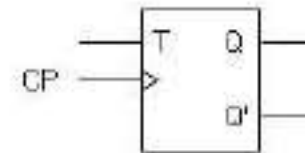
- Figure 18. T flip-flop: (a) circuit; (b) truth table; (c) logic symbol.

T TYPE FLIP-FLOP :

- This is a much simpler version of the J-K flip flop. Both the J and K inputs are connected together and thus are also called a single input J-K flip flop. When clock pulse is given to the flip flop, the output begins to toggle. Here also the restriction on the pulse width can be eliminated with a master-slave or edge-triggered construction. Take a look at the circuit and truth table is shown in figure.



(a) Logic diagram



(b) Graphical symbol

Q	T	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

(c) Transition table

Clocked T flip-flop

THANK you